

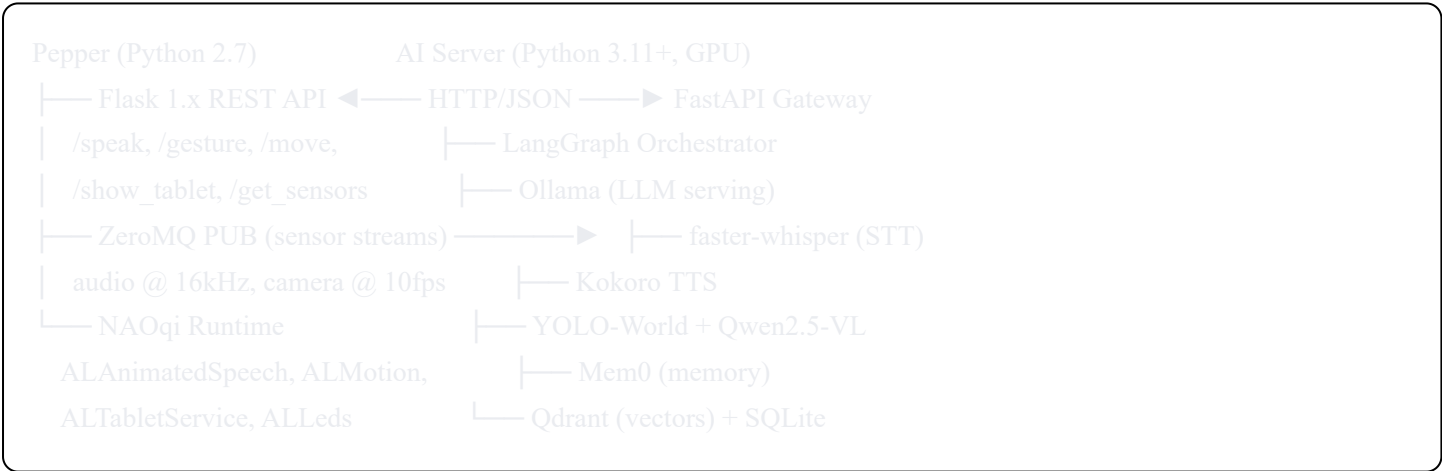
Making Pepper robot brilliant with modern AI

A SoftBank Pepper robot can be transformed from a scripted interaction device into a genuinely intelligent, adaptive, multimodal assistant by layering modern AI systems on top of a split-brain architecture — a Python 2.7 NAOqi controller on the robot communicating with a Python 3.11+ AI server running LLMs, vision models, speech pipelines, and persistent memory. The total hardware cost for the companion AI server is roughly **\$800–1,200** using entirely open-source software. This report provides the complete technical blueprint: specific models, frameworks, architecture patterns, and implementation priorities.

The split-brain architecture that makes everything possible

The foundational design separates concerns cleanly. Pepper's NAOqi SDK (Python 2.7) handles motor control, sensors, LEDs, and the tablet. A separate GPU-equipped server (Python 3.11+) runs all AI processing. They communicate over a local network using **REST APIs** (Flask 1.x on the robot side, FastAPI on the server) for commands, and **ZeroMQ PUB/SUB** for real-time sensor streaming (audio frames, camera frames).

This pattern is battle-tested. The **PepperChat** project (University of Skövde) proved the ZeroMQ bridge approach works reliably for ChatGPT integration. [GitHub](#) [github](#) The **Leolani project** (VU Amsterdam) runs a Flask server directly on Pepper exposing [/audio](#), [/video](#), and [/text](#) endpoints. [GitHub](#) For this build, combine both approaches:



The recommended companion server hardware: any modern 8-core CPU, **NVIDIA RTX 4060 Ti 16GB** (~\$400) or RTX 4090 24GB for maximum headroom, 32GB RAM, and a 500GB NVMe SSD. This comfortably runs a 7B language model, a 7B vision model (quantized), real-time speech recognition, and text-to-speech simultaneously.

Advanced RAG turns Pepper's documentation into deep expertise

Beyond basic LangChain RAG, a layered retrieval architecture transforms Pepper's PDF collection into a queryable knowledge system that handles everything from "What is Pepper's maximum speed?" to "How do the sonar sensors relate to the ALMotion obstacle avoidance API?"

PDF processing starts with Docling (IBM Research, MIT license), which achieves **97.9% accuracy** on complex table extraction ([Procycons](#)) and produces structured Markdown preserving document hierarchy. ([The New Stack](#)) For pages with schematics and diagrams, supplement with LlamaParse (LlamaIndex's cloud parser, free tier of 1,000 pages/day). Use a VLM like Qwen2.5-VL to generate text descriptions of diagrams, storing these alongside extracted text.

Chunking should be hierarchical with contextual augmentation. Docling's built-in Hybrid Chunker respects document structure (sections, tables, lists). Layer on Anthropic's Contextual Retrieval technique — prepend an LLM-generated context snippet to each chunk before embedding. This single technique reduces retrieval failures by **49%** when combined with hybrid search, and **67%** when reranking is added. Use three chunk levels: 2048-token section parents, 512-token paragraph children, and standalone table chunks.

Hybrid search is essential for technical documentation because Pepper PDFs contain model numbers, API names (ALMotion, ALTextToSpeech), and error codes that need exact keyword matching alongside semantic search. **Qdrant** is the recommended vector database — it natively supports dense + sparse (BM25) + ColBERT search in a single query, ([LiquidMetal AI](#)) ([Qdrant](#)) runs as a lightweight Docker container, and its Rust core keeps memory usage low. ([Airbyte](#)) Set the semantic/keyword balance at roughly 0.6–0.7 favoring vectors, with strong keyword weighting for technical terms.

For cross-document reasoning, deploy an Agentic RAG architecture using LlamaIndex. Create separate indexes per document type — hardware manuals, API documentation, troubleshooting guides — each wrapped as a tool. A ReAct agent routes queries to the appropriate index, ([LlamaIndex](#)) and a ([SubQuestionQueryEngine](#)) decomposes complex multi-document queries automatically. Layer **Corrective RAG (CRAG)** on top: a document grader scores retrieved passages for relevance, and if quality is too low, the system reformulates the query or searches a broader index. ([DataCamp](#)) ([Nb-Data](#))

For the most complex cross-document entity queries ("What software APIs control the motors described in the hardware manual?"), **Microsoft's GraphRAG** builds a knowledge graph with community summaries that excel at connecting disparate information. ([GitHub](#)) GraphRAG indexing costs roughly \$0.50–2.00 per document with GPT-4-class models, but the resulting community summaries handle holistic questions that standard RAG cannot. ([Microsoft](#))

The complete RAG pipeline flows: Docling → Hierarchical Chunking → Contextual Augmentation → Dual Embedding (dense + BM25) → Qdrant → Agentic Router → CRAG Evaluation → Reranking → LLM Generation.

Multimodal perception gives Pepper eyes, ears, and understanding

The multimodal stack runs three parallel pipelines — speech, vision, and audio understanding — fused through a late-fusion orchestrator that feeds structured context to the reasoning LLM.

Speech recognition uses faster-whisper (CTranslate2 backend), which runs **4x faster** than OpenAI's original Whisper with identical accuracy. The ([large-v3-turbo](#)) model achieves near-best accuracy at much faster speeds, ([OpenAI](#)) requiring ~3GB VRAM. Pair it with **Silero VAD v6** — a 1.8MB model that processes 30ms audio chunks in under 1ms on CPU, ([Medium](#)) with **87.7% true positive rate** at 5% false positive rate across 6,000+

languages. [Picovoice](#) The speech pipeline: continuous audio capture at 16kHz → Silero VAD detects speech onset → audio buffered with 300ms padding → faster-whisper batch transcription → text output in under 500ms.

Text-to-speech uses Kokoro-82M, the current leader on HuggingFace's TTS Arena. At just 82 million parameters, [BentoML](#) it generates speech at **40–70ms latency** on GPU and 3–11x real-time on CPU, [Unreal Speech](#) producing natural-sounding output across 48 voices in 8 languages. [Unreal Speech](#) For emotional expressiveness, switch to **Chatterbox-Turbo** (Resemble AI, 350M params, MIT license) which offers an emotion exaggeration control dial [BentoML](#) and beats ElevenLabs in blind preference tests. [Ocdevel](#) Implement sentence-level streaming — start TTS on the first sentence while the LLM generates the rest — to mask generation latency.

Vision processing operates at two speeds. Real-time detection runs **YOLO-World** (Tencent) at 30+ FPS for open-vocabulary object detection — it can detect any object via text prompt without custom training, [Roboflow](#) which is ideal for a robot encountering unpredictable environments. **MediaPipe** (Google) runs on CPU for face detection (468 landmarks), hand tracking, and pose estimation. Emotion recognition uses EfficientNetV2 [arXiv](#) or the DeepFace library for 7-class facial emotion classification. Every 1–5 seconds (or on demand), **Qwen2.5-VL-7B** (INT4, ~6–8GB VRAM) provides deep scene understanding with spatial reasoning and visual grounding via bounding boxes. This model rivals GPT-4o on document understanding benchmarks while fitting on a single consumer GPU. [Label Rer](#)

The GPU memory budget on a 24GB card allocates roughly: VLM 6GB, YOLO 2GB, STT 3GB, TTS 0.5GB, face/emotion 0.5GB, and sound events 0.3GB — totaling ~12.3GB with comfortable headroom.

Non-speech audio understanding rounds out perception. **PANNs** (Pretrained Audio Neural Networks) or **YAMNet** classify 500+ sound categories (doorbells, alarms, laughter, clapping) with minimal compute. Speaker identification via **pyannote-audio** enables the robot to know *who* is speaking. Vocal emotion detection using wav2vec 2.0 fine-tuned on emotion datasets provides tone-of-voice sentiment alongside facial expression analysis.

Memory systems let Pepper remember, learn, and personalize

Persistent memory transforms Pepper from a stateless chatbot into a relationship-building assistant. The recommended architecture uses a three-tier hierarchy with two complementary frameworks.

Mem0 [arXiv](#) (Apache 2.0, [DataCamp](#) \$24M Series A, [DataCamp](#) 41,000+ GitHub stars) [TechCrunch](#) serves as the production memory layer. It automatically extracts salient facts from conversations, consolidates conflicting information, [VentureBeat](#) applies decay to outdated memories, and retrieves relevant context via semantic search. [Built In San Francisco](#) It achieves **26% relative improvement** over OpenAI Memory in benchmarks with **91% lower latency** and **90%+ token cost savings** versus full-context approaches. [arXiv](#) Integration is three lines of code: [PR Newswire](#)

```
python
```

```
from mem0 import Memory
memory = Memory()
memory.add(conversation_messages, user_id="visitor_42")
relevant = memory.search("dietary preferences", user_id="visitor_42")
```

Letta ([HPCwire](#)) (the evolution of MemGPT, \$10M seed funding) provides deeper stateful agent behavior through virtual context management. Its architecture mirrors OS memory management: **core memory** (in-context "RAM" with editable persona and user blocks), **archival memory** (vector-backed "disk" for long-term storage), and **recall memory** (searchable conversation history). The LLM itself acts as the memory manager, deciding what to save, update, or retrieve via tool calls ([Letta](#)) like [core_memory_append](#) and [archival_memory_search](#). Letta's **sleep-time compute** feature (December 2025) performs asynchronous memory consolidation between interactions — compressing, deduplicating, and updating beliefs without blocking conversations.

The three tiers work together as follows. **Working memory** holds the current conversation context and recent perception data. **Session memory** stores LLM-generated summaries of each interaction session. **Long-term memory** maintains user profiles in a knowledge graph (capturing relationships: user → preferences, user → knowledge level, user → interaction history) plus Mem0's vector store for archival facts.

User personalization builds naturally from this foundation. Track each user by face recognition (linking to their profile ID), accumulate their preferences, knowledge level, interaction patterns, and emotional tendencies. Adapt speech speed, vocabulary complexity, topic suggestions, and proactive behaviors accordingly. When a returning visitor approaches, Pepper can greet them by name and reference their last conversation — a capability that dramatically improves perceived intelligence.

Learning from feedback happens through the Reflexion pattern: after each interaction, the system evaluates outcomes (did the user seem satisfied? did they correct information? did they re-ask a question?), stores reflections in memory, and adjusts future behavior. Explicit corrections ("No, I prefer tea, not coffee") trigger immediate memory updates. ([VentureBeat](#)) This creates a system that genuinely improves over time without any model retraining.

Agentic orchestration with LangGraph as the robot's brain

LangGraph is the recommended orchestration framework for Pepper because it models robot behaviors as stateful directed graphs with cycles, conditional routing, and persistence ([GitHub](#)) — exactly what physical robot control demands.

Define Pepper's behavior as a state machine with explicit states ([Sider](#)) (idle, listening, thinking, speaking, performing_task, error_recovery) as graph nodes and sensor inputs, user requests, and safety checks as conditional edges. LangGraph's built-in checkpointing means the robot doesn't lose state on restart, and its human-in-the-loop support enables remote operator override. ([GitHub](#)) ([TrueFoundry](#))

```
python
```

```

class PepperState(TypedDict):
    messages: list
    robot_action: str
    emotion_detected: str
    person_detected: bool
    memories: list
    current_scene: str

graph = StateGraph(PepperState)
graph.add_node("perceive", perceive_environment)
graph.add_node("retrieve_memory", fetch_memories)
graph.add_node("think", llm_reasoning)
graph.add_node("decide_action", plan_robot_action)
graph.add_node("validate_safety", safety_check)
graph.add_node("execute", send_to_robot)
graph.add_node("update_memory", store_memories)

```

Tool calling connects the LLM to physical actions by defining robot capabilities as typed Python functions with clear docstrings and safety constraints. The LLM never generates raw motor commands — it selects from a curated action library mapped to NAOqi APIs:

- `say(text, emotion)` → `ALAnimatedSpeech` with gesture tags
- `move_to(x, y, theta)` → `ALMotion` with boundary validation
- `look_at(target)` → `ALTracker` for social gaze
- `gesture(name)` → `ALAnimatedSpeech` animation tags
- `show_on_tablet(content)` → `ALTabletService`
- `set_leds(group, color)` → `ALLeds` for emotional expression

Use **Instructor** (Pydantic-based structured output extraction) to guarantee the LLM always returns valid, typed robot commands. For additional safety, **Outlines** provides constrained decoding that guarantees valid JSON/schema output during generation, not just post-hoc validation.

Planning uses a hierarchical approach. For reactive interactions (responding to greetings, answering questions), the **ReAct** pattern works well — Thought → Action → Observation → repeat. (Apideck) For complex multi-step tasks ("organize a trivia game for three people"), switch to **Plan-and-Execute**: the LLM frontloads a structured plan, validates it against physical constraints, then executes step-by-step with re-planning on failures. The **BrainBody-LLM** pattern maps naturally: one LLM plans the interaction flow (brain), another generates specific robot commands (body), (Techxplore) achieving **84% success rate** on manipulation tasks in research benchmarks. (Techxplore)

Behavior trees via py_trees provide the execution layer between AI decisions and robot actions. The LLM generates structured JSON describing tree topology, which is parsed into `py_trees` composites (Sequence, Selector, Parallel) at runtime. The **Dendron** library purpose-built for LLM+BT integration offers direct DSPy support for optimized prompting within tree nodes. (arXiv)

Safety guardrails are non-negotiable for physical AI

When an LLM controls a physical robot, safety requires multiple independent layers rather than trust in any single mechanism.

RoboGuard (UPenn/Kumar Robotics, 2025) is the state-of-the-art framework, reducing unsafe plan execution from **92% to below 2.5%** while maintaining performance on safe plans. (arXiv) It uses a separate "root-of-trust" LLM to generate temporal logic safety constraints, then a control synthesis module resolves conflicts between safety specifications and the planning LLM's output. (OpenReview) The project is open-source on GitHub.

(GitHub)

For Pepper specifically, implement five layers:

- **Action whitelist:** The LLM can only call pre-approved functions from the primitives library. No raw joint angle commands ever pass through.
- **Parameter bounds:** Every function enforces hard limits — maximum speed (0.3 m/s), maximum travel distance, allowed gesture set, navigation boundaries.
- **Command validation layer:** A safety validator on the Python 3 side checks every command before forwarding to the robot, rejecting forbidden zones, excessive speeds, or disallowed action combinations.
- **Hardware watchdog:** If the ZeroMQ/REST heartbeat between server and robot fails for more than 2 seconds, the robot enters a safe idle posture. A physical emergency stop button operates independently of software.
- **Timeout protection:** If the LLM takes too long to respond, the robot enters an idle animation rather than freezing, maintaining natural-seeming behavior.

A critical first step is disabling Pepper's **ALAutonomousLife** service, which otherwise conflicts with custom AI control. Always start the robot controller with (life.setState("disabled")).

Gesture generation and emotional expression bring Pepper to life

Natural gesture-speech synchronization dramatically improves perceived intelligence. The simplest effective approach embeds animation tags directly in ALAnimatedSpeech text: (^start(animations/Stand/Gestures/Hey_1) Hello!). Pepper ships with **100+ built-in animations** across categories (Gestures/, Emotions/, Reactions/, Waiting/), accessible via (ALAnimationPlayer.getAnimationList()).

The LLM should output structured responses that include both speech text and gesture annotations. Include the available animation names in the system prompt and have the LLM return JSON specifying gesture timing, animation selection, and LED colors mapped to emotional state (green eyes for happy, blue cycling for thinking, yellow for confused, white pulsing for listening). Recent research from Frontiers in Robotics (2025) demonstrates that **gesture generation is much faster than text generation**, so co-speech gestures can be planned as text streams without added latency. (Frontiers)

For more sophisticated gesture generation, **GesGPT** uses the LLM itself for semantic analysis of speech text, mapping semantic tags to gesture categories. [ResearchGate](#) **LLM Gesticulator** (2024) tokenizes motion data with VQ-VAE and trains an LLM to predict motion tokens from audio — following a scaling law where bigger models produce better gestures. [arXiv](#)

The recommended model stack and why each piece was chosen

Primary LLM: Qwen 2.5-7B-Instruct — Apache 2.0 license, best-in-class multilingual support (29 languages), excellent tool/function calling, strong reasoning. The 7B size hits the sweet spot for single-GPU deployment at ~25 tokens/second on an RTX 4060 Ti with Q4_K_M quantization. Use **Qwen2.5-3B** as a fast fallback for simple interactions (sub-second responses).

Primary VLM: Qwen2.5-VL-7B (INT4) — Rivals GPT-4o on document understanding, features native dynamic resolution, [Label Rer](#) visual grounding with bounding boxes, and video understanding. Runs at ~6–8GB VRAM quantized. [deepwiki](#) For ultra-lightweight visual Q&A, **Moondream2** (1.6B params, ~2GB) provides basic scene descriptions in under a second.

Inference serving: Ollama for development and single-robot production. Instant model switching, OpenAI-compatible API, GGUF quantization support. [LearnOpenCV](#) Wrap with **LiteLLM** for a unified interface that enables seamless fallback to cloud APIs (GPT-4o, Claude) when local inference is insufficient.

Vector database: Qdrant (self-hosted Docker) — Rust-based, fast, supports hybrid search natively, excellent metadata filtering [LiquidMetal AI](#) for document type and section routing.

Databases: SQLite via aiosqlite for conversation history, user profiles, and interaction metadata. Zero configuration, perfect for a single-robot system.

Observability: Langfuse (open-source) or LangSmith for tracing all LLM calls, viewing chain execution, and tracking costs. Pair with structlog for structured JSON logging of every AI decision.

The complete Docker Compose deployment runs three containers — Ollama (GPU-enabled), Qdrant, and the AI server application — alongside a Flask 1.x bridge running directly on Pepper.

Implementation roadmap from basic to brilliant

The system should be built in phases, each delivering visible capability improvements:

Phase 1 — Conversational foundation (weeks 1–3): Set up the split-brain communication bridge (Flask on Pepper, FastAPI on server). Deploy Ollama with Qwen2.5-7B. Implement the speech pipeline (Silero VAD → faster-whisper → LLM → Kokoro TTS → Pepper speaker). This alone gives Pepper natural, intelligent conversation.

Phase 2 — Knowledge and RAG (weeks 3–5): Process Pepper PDFs with Docling. Build the Qdrant vector index with hierarchical chunking and contextual augmentation. Implement hybrid search and the basic agentic RAG router with LlamaIndex. Pepper can now answer detailed technical questions about itself.

Phase 3 — Memory and personalization (weeks 5–7): Integrate Mem0 for persistent memory across sessions. Add face recognition for user identification. Implement user profiling and the Reflexion pattern for learning from interactions. Pepper now remembers visitors and adapts.

Phase 4 — Vision and multimodal understanding (weeks 7–9): Deploy YOLO-World for real-time object detection. Add Qwen2.5-VL for on-demand scene understanding. Integrate MediaPipe for gesture recognition and emotion detection. Build the late-fusion orchestrator. Pepper can now see and understand its environment.

Phase 5 — Agentic behaviors and safety (weeks 9–12): Implement the full LangGraph orchestration with behavior trees. Build the safety validation layer. Add complex task planning (Plan-and-Execute for multi-step requests). Implement gesture-speech synchronization. Deploy the testing pipeline (mock server → qiBullet simulation → physical robot).

Phase 6 — Advanced capabilities (ongoing): Add GraphRAG for cross-document reasoning. Implement sound event detection. Deploy Letta for deep stateful agent behavior with sleep-time memory consolidation. Build domain-specific applications (tutoring modules, research demo modes, visitor guide behaviors).

Conclusion

The path from a stock Pepper robot to a genuinely intelligent assistant is now well-paved with open-source tools. The critical architectural insight is the split-brain pattern — it completely decouples the aging NAOqi SDK from modern AI capabilities, meaning every future model improvement drops in without touching robot control code. **Qwen2.5-7B + Qwen2.5-VL-7B + faster-whisper + Kokoro TTS** form a multimodal stack that fits on a single consumer GPU while delivering near-frontier-model performance. **Mem0 + Letta** solve the persistent memory problem that makes the difference between a chatbot and a relationship-building assistant.

[TechCrunch +2](#) **LangGraph** provides the stateful orchestration that physical robot control demands, [Turing](#) [Leanware](#) while **Qdrant-backed agentic RAG** with contextual chunking turns static PDFs into queryable expertise. [Digital One Agency](#) The entire software stack costs nothing in licensing, and the hardware investment is under \$1,200. The biggest remaining challenge is not technical capability but integration discipline — carefully layering these systems with proper safety guardrails, thorough simulation testing, and incremental deployment ensures a robot that is not just smart, but reliably and safely smart.